Relatório da Fase 1 – Laboratórios de Informática 3

Grupo 07 – Bruno Santos (a113332), Salvador Fonseca (a113287), Nuno Correia (a115229)

1- Introdução

Este trabalho prático, desenvolvido no âmbito de Laboratórios de Informática 3, consiste numa aplicação em C que carrega um conjunto de dados de um marketplace (clientes, vendedores, produtos, encomendas e linhas de encomenda), valida-os e responde a consultas sobre eles. Na Fase 1, o programa carrega os cinco ficheiros CSV, rejeita as entradas inválidas e executa as queries Q1 a Q3 a partir de um ficheiro de comandos.

O trabalho assenta em três ideias: uma arquitetura em camadas com módulos de responsabilidade única; uma validação em cascata, em que um registo que referencia um registo inválido é também inválido; e um programa de testes que compara automaticamente os resultados obtidos com os esperados.

Destacamos duas decisões. Primeiro, separámos quem pode escrever dados de quem apenas os lê: só a carga mexe nos gestores, enquanto as queries recebem os dados só para leitura. Se uma query tentar alterar algo onde não deve, o compilador avisa, e a regra do grupo de compilar sem avisos impede que essa alteração entre. Segundo, corrigimos a meio da fase a forma como as linhas guardavam os identificadores, o que reduziu o pico de memória em cerca de 30%.

2- Sistema

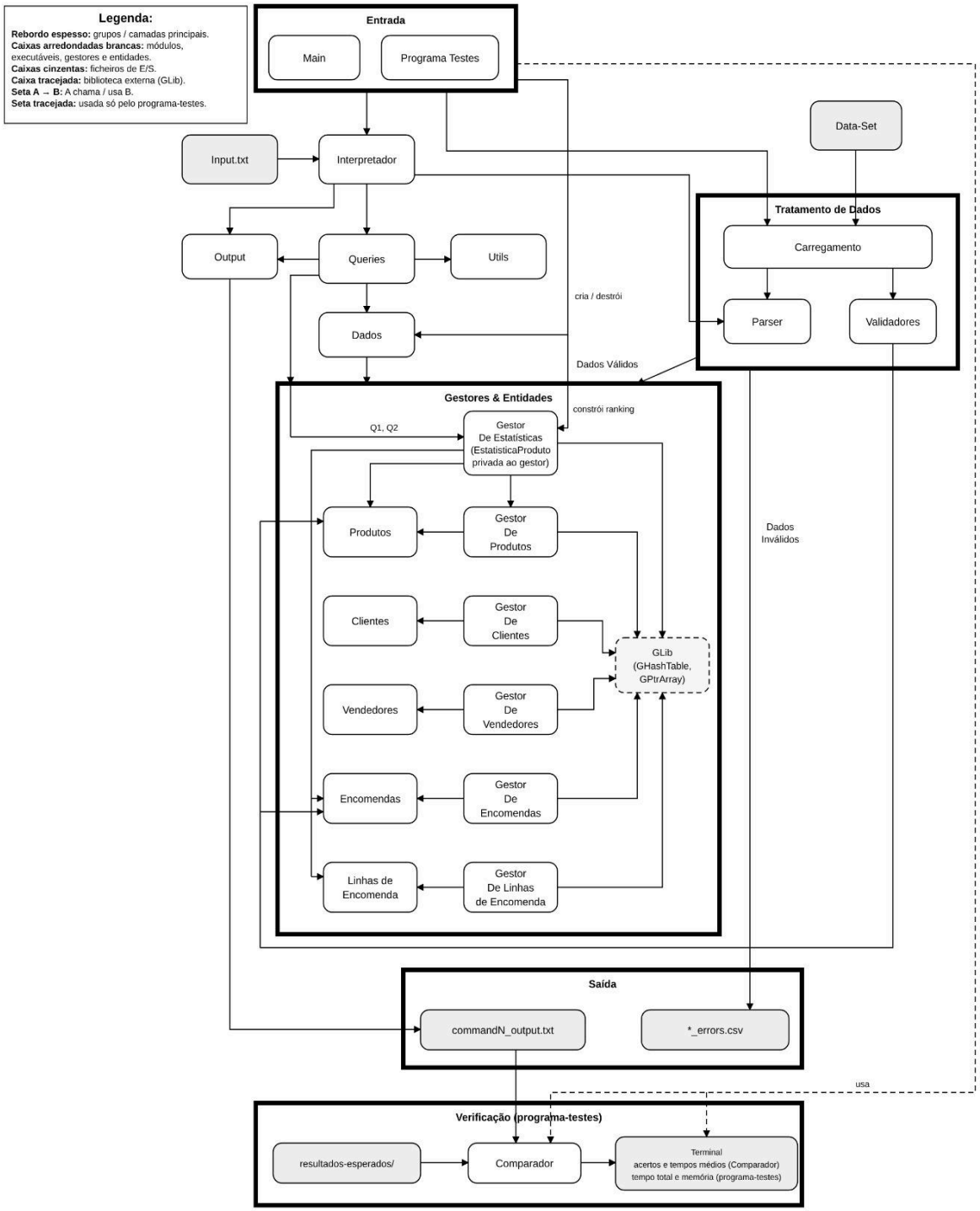


Figura 1: Arquitetura implementada

O sistema divide-se em camadas, em que cada módulo depende apenas dos que se encontram abaixo dele, como se vê na Figura 1. Esta organização limita o impacto de qualquer alteração: acrescentar uma nova query, por exemplo, afeta poucos módulos.

Na camada de entrada, os executáveis main e main_testes limitam-se a orquestrar a execução: criam a estrutura de dados, invocam a carga de cada ficheiro, invocam o interpretador e libertam a memória no final. É nestes executáveis que se define a ordem de carga (vendedores, clientes, produtos, encomendas e linhas), porque cada entidade referencia a anterior. Depois da carga, constroem também a ordenação guardada no gestor de estatísticas. O main_testes acrescenta a medição do tempo total e da memória de pico e invoca o comparador.

O tratamento dos dados é feito pelo módulo carregamento, que lê cada ficheiro CSV e recorre ao parser para separar os campos de cada linha e aos validadores para verificar a sintaxe dos valores. As verificações lógicas (por exemplo, se o vendedor de um produto existe) são feitas no próprio carregamento, consultando os gestores já carregados. Os registos válidos são inseridos nos gestores e os inválidos são escritos nos ficheiros *_errors.csv. Como um registo inválido nunca chega a ser inserido, qualquer registo que o referencie falha a pesquisa e é também inválido, o que implementa a regra de cascata.

Na camada de armazenamento, cada entidade tem um gestor próprio, responsável por guardar os registos, pesquisá-los por identificador e libertar a memória que ocupam. As entidades são tipos opacos, expostos apenas através de um construtor, de um destrutor e de funções de acesso. Assim, nenhum módulo exterior depende da sua representação interna.

Por fim, o interpretador lê o ficheiro input.txt, escolhe o separador e despacha cada comando para a query correspondente. As queries consultam os dados através do módulo dados e, quando precisam de uma entidade em concreto, dos gestores respetivos. O resultado é escrito por meio do módulo output, que produz os ficheiros commandN_output.txt.

Cada módulo existe para isolar uma responsabilidade única. As entidades cliente, vendedor, produto, encomenda e linha_encomenda representam os conceitos do problema e escondem a sua estrutura interna. Os gestores gestor_clientes, gestor_vendedores, gestor_produtos e gestor_encomendas guardam e pesquisam cada entidade por identificador e são donos da memória dos registos, o que torna claro onde cada bloco é libertado. O gestor_linhas_encomenda agrupa as linhas por encomenda, o que permite obter todas as linhas de uma encomenda com uma única pesquisa (necessário na Q3) e verificar se o par (encomenda, produto) já existe, rejeitando linhas repetidas.

O gestor_estatisticas existe para que as queries não recalculem o que é sempre igual. Durante a carga das linhas, regista as unidades vendidas e a receita de cada produto, excluindo as encomendas canceladas. Depois da carga, constrói a ordenação dos produtos por unidades vendidas e guarda-a. Assim, a Q1 reduz-se a uma consulta de valores já

calculados, e a Q2 reutiliza a mesma ordenação em todos os pedidos, em vez de ordenar os produtos de cada vez.

O módulo dados agrega todos os gestores numa única estrutura, entregue às queries apenas para leitura e à carga e aos executáveis com permissão de escrita, o que protege os dados de alterações indevidas. O parser e os validadores evitam repetir a separação de campos e as regras sintáticas do enunciado, e podem ser testados isoladamente. O parser é também usado pelo interpretador para separar os argumentos de cada comando. O carregamento liga estes módulos, aplica as validações lógicas e a cascata e escreve os ficheiros de erro.

O interpretador torna as queries independentes do formato do ficheiro de comandos: lê cada linha, identifica a query, o separador a usar e despacha-a. O módulo output garante que nenhuma query manipula ficheiros diretamente e que o separador correto (; ou =) é sempre aplicado. O módulo utils reúne a conversão de cêntimos para texto, partilhada entre queries.

O módulo queries contém as três consultas. A Q1 devolve, a partir de um identificador de produto, os valores guardados no gestor de estatísticas, a ficha do produto e o nome do vendedor, e uma linha vazia se o identificador não existir. A Q2 devolve os N produtos mais vendidos, com desempate por identificador e filtro opcional de categoria, a partir da ordenação guardada pelo gestor de estatísticas. A Q3 compõe a resposta a partir da encomenda, do cliente e das suas linhas.

Para a verificação, o comparador compara os resultados obtidos com os esperados, indica a primeira linha em que diferem e calcula os acertos por query, e o programa-testes acrescenta o tempo total e a memória de pico. Os testes unitários, executados com make test, permitem detetar regressões nos módulos que cobrem.

3- Discussão

A decomposição seguiu um critério simples: cada módulo tem uma única razão para mudar. A carga dos dados muda se o formato dos ficheiros mudar, os gestores mudam se mudar a forma de guardar os registos, e as queries mudam se o enunciado pedir uma nova consulta. Separar estas responsabilidades faz com que uma alteração fique contida num número reduzido de módulos. Se aparecesse uma Q4, por exemplo, só seriam alterados o queries, o interpretador e o comparador do programa-testes, enquanto a carga, a validação e os gestores permaneceriam iguais.

Cada módulo tem um .h com a interface documentada e um .c com a implementação. As struct das entidades e dos gestores só são definidas no .c, e as estruturas internas (por exemplo, as estatísticas de cada produto e as posições do ranking) e as funções auxiliares são static ou privadas ao .c. As dependências seguem um só sentido: o dados conhece os gestores, mas nenhum gestor conhece o dados; os gestores conhecem as entidades, mas nenhuma entidade conhece um gestor; e o gestor de estatísticas consulta o gestor de produtos sem que o contrário aconteça. Por isso não há dependências circulares.

Depois do Checkpoint 1 fizemos duas mudanças. A primeira foi separar quem valida de quem guarda. No desenho do Checkpoint 1, os validadores estavam no caminho dos dados válidos. Passámos a ter o validador a limitar-se a responder “válido” ou “inválido”, e é o módulo carregamento que entrega os registos válidos aos gestores e escreve os inválidos nos ficheiros *_errors.csv. Pelo mesmo critério, retirámos o processamento dos ficheiros de dentro das entidades, que ficaram a conter apenas os seus dados; assim, as regras de formato podem ser testadas isoladamente, sem gestores nem ficheiros, e o parser e os validadores são reutilizados pelas cinco cargas. A segunda foi concentrar o acesso aos gestores no módulo dados, entregue às queries apenas para leitura, o que impede que uma query altere os dados carregados.

A cascata exigida pelo enunciado, em que um registo que referencie outro inválido ou inexistente é também inválido, resulta da ordem de carga: vendedores e clientes, produtos, encomendas e linhas. Como um registo inválido nunca é inserido, qualquer registo que o referencie falha a pesquisa no gestor e é rejeitado. Por exemplo, uma encomenda com um cliente inexistente é inválida e as suas linhas também. Sem esta regra, linhas de encomendas rejeitadas contariam para as estatísticas de vendas e distorceriam a Q1 e a Q2.

Qualidade e testes. A garantia de qualidade e a prevenção de regressões assentam numa regra estrita adotada pela equipa: um módulo só é considerado concluído depois de compilar sem avisos no ambiente Docker, passar na totalidade dos testes unitários, não apresentar fugas de memória no Valgrind, ter a documentação em ficheiros header devidamente estruturada com as tags necessárias e obter a aprovação de outro elemento do grupo num Pull Request.

O programa de testes executa o mesmo fluxo do programa principal, compara cada ficheiro de saída com o resultado esperado e indica, por query, a quantidade de casos corretos e a primeira linha onde ocorre uma divergência, contando cada variante S na query do mesmo número. Este teste é efetuado tanto no dataset normal como no dataset de erros, porque as queries só dão o resultado certo se os validadores rejeitarem exatamente as linhas certas. Cada submissão na plataforma foi comparada com o resultado local para detetar diferenças.

Os testes unitários (make test, 46 verificações) cobrem os gestores de produtos, encomendas, linhas de encomenda e estatísticas, nos casos normal, limite e de erro, e o validador de identificadores. Na gestão de memória, o Valgrind reporta 0 erros e nenhum bloco perdido; os cerca de 16 KB que restam acessíveis no fim pertencem à GLib.

Com o dataset oficial com erros e os resultados esperados fornecidos, o programa-testes obteve Q1 10/10, Q2 10/10 e Q3 10/10, e os cinco ficheiros *_errors.csv são iguais, byte a byte, aos esperados. O tempo e a memória de pico medidos pelo programa de testes só serão analisados na Fase 2.

4- Conclusão

A Fase 1 mostrou-nos que dividir o programa por responsabilidades bem definidas facilita imenso o trabalho em grupo e a correção de erros. Retirar o processamento de ficheiros das entidades e separar quem valida de quem guarda tornou cada parte testável isoladamente.

O que correu melhor foi a validação em cascata, que funciona do início ao fim de forma simples, dependendo apenas da ordem de carregamento dos dados, sem código dedicado a propagar erros. Todo o desenvolvimento decorreu através de branches e pull requests com revisão obrigatória por um colega, nunca diretamente na main.

O que correu menos bem foi o desenho inicial: tínhamos os dados a passar diretamente pelos validadores e não contávamos com um gestor de estatísticas dedicado, o que nos obrigou a refatorizar a arquitetura, o diagrama e o código. A dificuldade mais concreta foi a memória. A meio da fase, cada linha de encomenda copiava com strdup os identificadores da encomenda e do produto: duas alocações por linha, em 11 milhões de linhas. Passámos a apontar para os identificadores que já pertencem à encomenda e ao produto, que vivem até ao fim do programa, e o pico de memória da carga completa desceu de cerca de 2,3 GB para 1,6 GB. Em retrospetiva, teríamos definido desde o início quem é dono de cada string.

Como melhoria para a Fase 2, prevemos reduzir ainda mais a memória usada, que na carga completa do dataset atinge cerca de 1,7 GB, já com as estatísticas e o ranking da Q2, concentrados sobretudo nas linhas de encomenda e nas encomendas. Planeamos guardar as linhas num array compacto, sem uma alocação por linha, e representar os identificadores por inteiros em vez de cópias de strings. Na Q2, um ranking por categoria permitiria responder ao filtro de categoria sem percorrer o ranking global. Por fim, prevemos dividir o módulo de queries por ficheiro, à medida que o número de queries aumentar.